

**A
RTRP PROJECT REPORT**

On

COLLEGE CHAT BOT

Submitted in partial fulfillment of the Requirements for the Award of the Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

BY

Team Member Names	Roll Number
G. ARAVIND	24N81A6261
K. DASHRATH	24N81A6224
K. UMESH	25N85A6205
J. RAM CHARN	25N85A6204
J. THARUN	24N81A6221

**Under the Guidance of
MRS. SANDHYA RANI**

Assistant Professor
Department of CSE (CS)

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CS)
SPHOORTHY ENGINEERING COLLEGE (AUTONOMOUS)**

{NAAC Accredited, Recognised by UGC, u/s 2(f) & 12(B)}
(Permanently Affiliated to JNTUH, Approved by AICTE, New Delhi, ISO 9001:2015 Certified)
Nadergul (Vill.), Near L.B. Nagar, Hyderabad, Telangana - 501510.

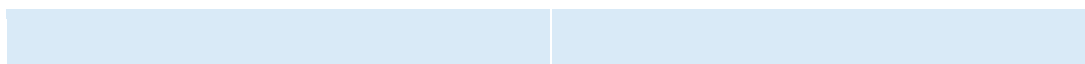
2024-2028

SPHOORTHY ENGINEERING COLLEGE
(AUTONOMOUS)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CS)

CERTIFICATE

This is to certify that this RTRP Project Report entitled COLLEGE CHAT BOT is a bonafide work carried out by (G. ARAVIND - 24N81A6261, K. DASHRATH - 24N81A6224, K. UMESH - 25N85A6205, J. RAM CHARN - 25N85A6204, J. THARUN - 24N81A6221) in partial fulfilment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering (Cyber Security) from Sphoorthy Engineering College (Autonomous), affiliated to Jawaharlal Nehru Technological University Hyderabad, during the Academic Year 2025-26 under our guidance and supervision.

The results embodied in this report have not been submitted to any other university or institute for the award of any degree or diploma.



ABSTRACT

College Chat Bot is a Flask-based academic assistance system designed to provide students with quick, reliable and centralised access to college information. The system replaces repeated manual enquiries at department offices with a responsive web interface where students can ask natural queries about attendance, examination schedules, placement drives, campus events and academic support services.

The proposed chatbot integrates an HTML, CSS and JavaScript frontend with a Flask application layer and a SQLite or MySQL relational database. A rule-based intent classification layer maps student messages to institutional data such as attendance percentage, upcoming examinations, notices, placement eligibility, event registrations and contact points for faculty support. The design supports administrator-managed datasets so that the information delivered by the bot remains accurate and easy to update.

The system is structured around secure login, query processing, knowledge management, notification display and administrative maintenance modules. Students interact through a compact chat window that works on desktop and mobile devices, while staff members can update timetables, placement announcements, event notices and frequently asked academic responses. Database tables are organised for users, queries, attendance records, exam schedules, placement updates, events and support tickets.

Testing was conducted across authentication, chatbot response accuracy, database operations, responsive user interface behaviour and administrator update workflows. The results indicate that the College Chat Bot improves query response time, reduces repetitive manual workload and offers an accessible academic support channel for students. Future enhancements include machine-learning-based intent recognition, multilingual support, voice input, integration with institutional ERP systems and proactive push notifications.

TABLE OF CONTENTS

Abstract	1
Table of Contents	2
List of Tables	3
1. Introduction	4
1.1 Background and Motivation	4
1.2 Problem Context	5
1.3 Project Overview	5
1.4 Scope of the System	6
1.5 Report Organisation	6
2. Literature Survey	7
2.1 Web-Based Academic Support Systems	7
2.2 Chatbot Technology in Education	8
2.3 Role of Flask in Academic Prototypes	8
2.4 Database-Driven Notification Systems	9
2.5 Research Gap	9
3. Problem Statement	10
4. Objectives	11
5. System Architecture	12
6. Technology Stack	14
7. System Modules	16
8. Database Design	19
9. Implementation	22
10. Testing	25
11. Results	28
12. Advantages	30
13. Future Scope	31
14. Conclusion	33
References	34

LIST OF TABLES

Table 5.1 - Three-Tier Architecture Components	13
Table 6.1 - Technology Stack Summary	15
Table 7.1 - System Module Summary	18
Table 8.1 - User Table Schema	20
Table 8.2 - Attendance Table Schema	20
Table 8.3 - Exam Schedule Table Schema	21
Table 8.4 - Placement and Event Table Schemas	21
Table 8.5 - Entity Relationship Summary	21
Table 10.1 - Authentication Module Test Cases	26
Table 10.2 - Chatbot Query Test Cases	27
Table 10.3 - Administration Module Test Cases	27
Table 11.1 - Overall Test Coverage Summary	29
Table 11.2 - Manual Enquiry vs Chatbot System Comparison	29

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Modern colleges handle a large amount of routine information every day. Students frequently ask about attendance status, exam dates, placement drives, event registration, academic resources, faculty contacts and department notices. In a manual environment, the same questions are repeated at classrooms, department offices, WhatsApp groups and notice boards, creating unnecessary workload for faculty and administrative staff.

The motivation for College Chat Bot is to create a single digital assistant that responds instantly to common academic queries. Instead of waiting for office hours or searching multiple notice channels, students can interact with a web-based chatbot that retrieves verified information from a structured database. This supports faster communication and improves the transparency of college services.

1.2 Problem Context

Information in many institutions is distributed across circulars, spreadsheets, departmental registers and informal message groups. When exam schedules or placement announcements are updated, students may not receive the latest version in time. Attendance information is also commonly requested, especially before internal examinations and eligibility checks.

Manual enquiry handling becomes difficult when several students ask similar questions at the same time. The lack of a searchable central interface causes delays, inconsistent responses and avoidable confusion. A chatbot system can reduce these issues by standardising answers and allowing administrators to update the underlying data in one place.

1.3 Project Overview

College Chat Bot is a full-stack web application consisting of a responsive chat frontend, a Flask backend and a relational database. The chatbot receives a student query, identifies its intent, fetches the required record and returns a clear answer. For example, the system can answer queries such as upcoming exam dates, attendance percentage, placement drive eligibility, event schedules and academic support contacts.

1.4 Scope of the System

- Student login and profile-based query access.
- Attendance lookup using roll number and subject-wise records.
- Exam schedule display for internal, practical and semester examinations.
- Placement updates including company name, eligibility, date and venue.
- Event notifications for technical, cultural, sports and club activities.
- Academic support through frequently asked questions and faculty contact references.
- Administrator screens for maintaining schedules, announcements and chatbot knowledge.

1.5 Report Organisation

The report is organised into fourteen chapters. The first four chapters introduce the project, survey related work, define the problem and state the objectives. Chapters five to eight describe architecture, technologies, modules and database design. Chapters nine to eleven explain implementation, testing and results. The final chapters list advantages, future scope, conclusion and references.

1.6 Need for a Chatbot in Campus Communication

A college environment contains several information sources: department notice boards, classroom announcements, faculty messages, training and placement circulars, examination cell schedules and student coordinator updates. Students often need an answer at the exact moment they are planning academic work, travel, event participation or placement registration. A chatbot is useful because it converts these distributed sources into a single conversational access point.

The chatbot approach is also familiar to students because messaging applications are already part of daily communication. Instead of navigating a complex portal, the student can type a question in ordinary language.

Even when the internal logic is rule-based, the interaction feels direct and reduces the number of clicks required to reach the information.

From the institution's point of view, the chatbot creates a repeatable method for publishing information. When a placement drive is added or an examination timetable is revised, the administrator updates one record and the bot immediately begins answering from the new data. This reduces dependency on repeated forwarding and prevents old notices from continuing to circulate informally.

- A single query interface reduces confusion caused by multiple notice channels.
- Database-driven answers improve consistency and reduce outdated responses.
- Query logs help departments identify information that students ask for repeatedly.

CHAPTER 2: LITERATURE SURVEY

2.1 Web-Based Academic Support Systems

Web-based academic management systems provide centralised access to student records, course material and institutional announcements. Learning management systems such as Moodle and ERP-style college portals have demonstrated the value of online academic communication. However, many such systems require students to navigate menus rather than ask direct questions. College Chat Bot focuses on conversational access to the same kind of information.

Web-based academic management systems provide centralised access to student records, course material and institutional announcements. Learning management systems such as Moodle and ERP-style college portals have demonstrated the value of online academic communication. However, many such systems require students to navigate menus rather than ask direct questions. The proposed system focuses on conversational access to the same kind of information.

2.2 Chatbot Technology in Education

Educational chatbots are increasingly used for answering admission queries, course FAQs and student support requests. Rule-based chatbots are suitable for institutional prototypes because the question categories are known in advance and the response data must be controlled by college staff. Natural language processing can later improve flexibility, but a database-backed rule system provides reliable results for attendance, schedules and notices.

Educational chatbots are increasingly used for answering admission queries, course FAQs and student support requests. Rule-based chatbots are suitable for institutional prototypes because the question categories are known in advance and the response data must be controlled by college staff. Natural language processing can later improve flexibility, but a database-backed rule system provides reliable results for attendance, schedules and notices.

2.3 Role of Flask in Academic Prototypes

Flask is a lightweight Python micro-framework widely used for academic and prototype systems. It supports routing, templates, form handling and integration with databases without imposing a large project structure. For College Chat Bot, Flask provides enough flexibility to implement chat APIs, administrator pages and authentication while keeping the codebase understandable for student developers.

the Flask framework is a lightweight Python micro-framework widely used for academic and prototype systems. It supports routing, templates, form handling and integration with databases without imposing a large project structure. For The proposed system, the Flask framework provides enough flexibility to implement chat APIs, administrator pages and authentication while keeping the codebase understandable for student developers.

2.4 Database-Driven Notification Systems

Notification systems in colleges often depend on notice boards and messaging applications. A relational database approach provides better consistency because each placement update, exam record or event notice is stored with structured fields such as date, department, eligibility and status. The chatbot can then generate responses from the latest active records rather than from hard-coded text.

Notification systems in colleges often depend on notice boards and messaging applications. A relational database approach provides better consistency because each placement update, exam record or event notice is stored with structured fields such as date, department, eligibility and status. The chatbot can then generate responses from the latest active records rather than from hard-coded text.

2.5 Research Gap

Existing college portals generally expose information through separate modules. Chatbot-based systems often answer static FAQs but do not integrate with attendance, exam, placement and event records. The proposed project addresses this gap by combining conversational query handling with structured academic data management in a compact Flask-based web application.

Existing college portals generally expose information through separate modules. Chatbot-based systems often answer static FAQs but do not integrate with attendance, exam, placement and event records. The proposed project addresses this gap by combining conversational query handling with structured academic data management in a compact the Flask framework-based web application.

2.6 Comparative Review of Existing Approaches

Traditional college websites are useful for publishing static information such as admission details, department profiles and academic calendars. However, they are not designed for personalised student responses. A student asking for attendance percentage or placement eligibility still requires a login-based data lookup, not a general website page.

Enterprise resource planning systems provide many academic features but are usually large, costly and difficult to customise for small departmental experiments. They often require dedicated administrators and may not expose a flexible conversational layer. For a student project, a lightweight web application provides a better balance between implementation feasibility and practical usefulness.

Messaging groups provide fast communication but weak information governance. Messages can be missed, deleted, misunderstood or forwarded without context. A chatbot does not replace official communication, but it provides a stable retrieval layer where students can confirm the current version of a schedule or notice.

2.7 Summary of Literature Findings

The survey indicates that web-based systems improve academic administration when they reduce manual dependency and make information available on demand. Role-based access control is important because students and administrators must see different functions. Database-backed notification systems are preferable to hard-coded responses because college information changes frequently.

The literature also shows that a conversational interface is effective only when the underlying knowledge base is accurate and maintained. Therefore, College Chat Bot gives equal importance to the administrator module and the student-facing chat interface.

CHAPTER 3: PROBLEM STATEMENT

The college requires a reliable digital system that can answer student queries related to attendance, examinations, placements, events and academic support without depending entirely on manual enquiry handling. The present communication process is fragmented across notice boards, staff messages, office visits and informal groups.

Students may miss important updates when they are absent, when a message is forwarded late or when a notice is revised after initial circulation. Faculty and administrative staff repeatedly answer the same questions, reducing time available for academic work. There is also no central record of the questions being asked by students, making it difficult to identify recurring information gaps.

The problem is therefore to design and implement a responsive chatbot application that authenticates users, interprets common academic queries, retrieves verified data from a database and returns concise answers through a web interface. The system should be easy for administrators to update and should support future integration with college ERP services.

3.1 Functional Requirements

- The system shall allow students to ask text-based queries through a browser.
- The system shall retrieve attendance, examination, placement and event information from database tables.
- The system shall provide an administrator interface for updating institutional data.
- The system shall maintain query history for analysis and service improvement.
- The system shall return meaningful fallback responses when a query is outside the available knowledge base.

3.2 Non-Functional Requirements

- Responsive user interface
- Fast query response
- Secure data access
- Simple maintainability
- Consistent academic formatting of responses

3.3 Stakeholder Analysis

Students are the primary users of the system. Their requirement is speed: they need immediate answers without waiting for faculty or office staff. They also require clarity because academic decisions such as exam preparation, placement registration and event participation depend on accurate dates and eligibility details.

Administrators and faculty members are secondary users. Their requirement is maintainability: they need simple forms for adding new schedules, correcting old notices and disabling expired announcements. A system that is difficult to update will quickly become unreliable, so the project emphasises compact administrative screens.

The department benefits from reduced repetitive enquiries and better visibility into common student questions. Query logs can reveal whether students are confused about attendance rules, exam dates, placement criteria or event registration processes.

3.4 Constraints and Assumptions

The prototype assumes that attendance records and notices are entered by an authorised administrator. Real-time synchronisation with an ERP is considered future scope. The rule-based chatbot assumes that common student queries can be grouped into known categories such as attendance, exams, placements, events and academic support.

The system also assumes that students use valid roll numbers and that personalised data is protected through login sessions. For deployment beyond a demonstration environment, HTTPS, strong password policies and periodic database backup are required.

CHAPTER 4: OBJECTIVES

The main objective of College Chat Bot is to simplify academic communication by providing a single web-based chatbot interface for student queries. The system aims to reduce manual enquiry load while improving the accuracy and availability of college information.

The project also aims to demonstrate the use of Flask, relational databases and responsive frontend technologies in a practical college support application.

- To implement student query handling for attendance, exams, placements, events and academic support.
- To design a responsive chat interface using HTML, CSS and JavaScript.
- To build a Flask backend that exposes secure routes and chatbot response APIs.
- To store institutional information in SQLite or MySQL tables with clear relationships.
- To support administrator maintenance of schedules, announcements and FAQ responses.
- To test the system for correctness, usability, performance and access control.

4.1 Measurable Project Goals

The project goals are defined in measurable terms so that implementation can be evaluated during testing. The chatbot should respond to common queries within a short time, should retrieve correct database records for known intents and should display a meaningful fallback response for unknown questions. The administrator should be able to add or update records without directly editing database files.

A second goal is usability. Students should be able to access the chat interface from a phone browser and understand the response without needing technical knowledge. Each response should be short enough for a chat bubble but complete enough to answer the question.

- Response accuracy for seeded test queries should be 100 percent.
- Administrator updates should be reflected in chatbot answers immediately after saving.
- The chat page should remain usable on common mobile and desktop screen widths.
- Personalised attendance data should only be shown after student login.

CHAPTER 5: SYSTEM ARCHITECTURE

The architecture of College Chat Bot follows a three-tier model. The presentation tier handles browser-based interaction, the application tier processes business logic through Flask routes and the data tier stores verified institutional records. This separation allows the user interface, chatbot logic and database schema to be developed and tested independently.

5.1 Architecture Components

Layer	Main Components	Responsibility
Presentation Tier	HTML, CSS, JavaScript, chat widget	Displays login pages, chat interface, notices and administrator forms.
Application Tier	Flask routes, chatbot engine, validation logic	Receives requests, identifies intent, validates user access and prepares responses.
Data Tier	SQLite/MySQL database	Stores users, attendance, exams, placements, events, FAQs and query logs.

5.2 Query Processing Flow

- The student enters a query in the chat interface.
- JavaScript sends the message to the Flask `/chat` endpoint using an asynchronous request.
- The backend normalises the query and compares it with known keywords and intent patterns.
- The relevant database table is queried using parameterised SQL or ORM methods.
- The response formatter converts the data into a concise student-friendly reply.
- The message and detected intent are saved in the query log for future analysis.

5.3 Security View

Authentication protects personalised records such as attendance. Administrator routes are restricted to authorised users so that only staff can change exam schedules, placement notices and event announcements. Input validation is applied before database operations to reduce malformed requests and SQL injection risk.

5.4 Data Flow Description

The data flow begins when the browser loads the chat page. Static files such as style sheets and JavaScript are loaded from the Flask static directory, while the initial HTML is rendered using a Jinja2 template. When the user enters a query, the frontend does not reload the page; it sends an asynchronous request containing the message text and the current session context.

The Flask route validates that the session is active and then passes the query to the chatbot service. The service normalises the text by converting it to lowercase, removing extra spaces and checking for keywords or patterns. Once an intent is detected, a database query is executed. For attendance, the query is filtered by the logged-in student's roll number. For general notices, active records are sorted by date.

The response object contains the answer text, the detected intent and a status flag. JavaScript receives this object and displays it as a bot message. The backend stores the query, response category and timestamp in a log table so that administrators can later review usage.

5.5 Architecture Benefits

The three-tier architecture improves maintainability because each layer can be modified independently. The frontend can be redesigned without changing database schema, while the database can move from SQLite to MySQL without rewriting the chat screen. Flask acts as the coordination layer that connects user actions to persistent records.

This architecture also supports gradual enhancement. A future machine-learning intent classifier can replace the keyword detector while keeping the same `/chat` API contract. Similarly, a mobile application can consume the same backend route if the college later chooses to provide a dedicated app.

CHAPTER 6: TECHNOLOGY STACK

The selected technology stack is intentionally lightweight and suitable for student project deployment. Flask reduces backend complexity, while HTML, CSS and JavaScript provide a browser-native user interface. SQLite is suitable for development and demonstration, whereas MySQL can be adopted for larger institutional deployments.

Technology	Purpose	Reason for Selection
Python	Backend programming	Readable syntax, strong library support and suitability for academic projects.
Flask	Web framework	Lightweight routing, template rendering and API support.
HTML5	Page structure	Defines semantic structure for login, chat and admin screens.
CSS3	Styling	Supports responsive layouts, typography and compact report-like UI elements.
JavaScript	Client interactivity	Handles asynchronous chat requests and dynamic response display.
SQLite/MySQL	Database	Stores structured college data with simple querying and migration options.
Bootstrap	Responsive UI	Provides grid, buttons, forms and mobile-friendly components.
Jinja2	Templates	Generates dynamic pages from Flask context data.

6.1 Development Environment

The system can be developed using Visual Studio Code, Python virtual environments and browser developer tools. During testing, SQLite simplifies setup because it does not require a separate database server. For production deployment, the same schema can be migrated to MySQL.

6.2 Deployment Considerations

A WSGI-compatible server can host the Flask application. Static assets are served from the application directory, while database backups are scheduled periodically. Environment variables should be used for secret keys and production database credentials.

6.3 Backend Technology Details

Python was selected because it is easy to read, widely taught and has strong support for web development. Flask provides a minimal core that allows students to understand the complete request-response cycle. Routes can be mapped directly to functions, making it easier to trace how a query moves from the browser to the database.

The backend can use SQLAlchemy or direct parameterised SQL. SQLAlchemy is preferable for larger versions because it represents tables as Python classes and reduces repetitive query code. Direct SQLite access may be sufficient for a small demonstration but requires careful handling of SQL parameters.

6.4 Frontend Technology Details

HTML provides the structural layout for login, chat and administrator pages. CSS controls the visual design, message bubble layout, table spacing and responsive breakpoints. JavaScript is responsible for submitting queries asynchronously and updating the chat conversation without refreshing the page.

The frontend is intentionally simple so that it loads quickly on college networks and mobile devices. The design avoids heavy graphics and focuses on readable typography, clear buttons and compact cards for notices.

6.5 Database Technology Details

SQLite is suitable for development because it stores data in a local file and requires no server setup. This helps during project demonstration and testing. MySQL is suitable for deployment when multiple users access the system at the same time and when formal backup, user privileges and remote administration are required.

The database layer is designed so that both options are possible. Table names, field types and relationships are kept conventional, making migration easier if the project moves from a prototype to an institutional pilot.

CHAPTER 7: SYSTEM MODULES

College Chat Bot is divided into modules so that each functional area has a clear responsibility. The modular design also supports easier debugging, testing and future extension.

Module	Functions	Users
Authentication	Login, logout, session handling and role validation	Students, Admin
Chatbot Engine	Intent detection, response generation and fallback handling	Students
Attendance Module	Subject-wise attendance lookup and eligibility messages	Students, Admin
Exam Module	Internal, practical and semester schedule display	Students, Admin
Placement Module	Company updates, eligibility and venue information	Students, Admin
Event Module	Technical, cultural and sports notifications	Students, Admin
Academic Support	FAQ responses, faculty contacts and support ticket routing	Students
Admin Module	CRUD operations for notices and master data	Admin

7.1 Student Query Handling

Student Query Handling is implemented as a focused service area within the chatbot. The module receives a detected intent, checks the logged-in student's context where required and returns information in a short response format suitable for chat display. Administrative maintenance screens keep the module data current.

Each module is designed to fail gracefully. If no active record is found, the bot replies with a clear message asking the student to contact the department office or check again after the data is updated.

7.2 Attendance Information

Attendance Information is implemented as a focused service area within the chatbot. The module receives a detected intent, checks the logged-in student's context where required and returns information in a short response format suitable for chat display. Administrative maintenance screens keep the module data current.

Each module is designed to fail gracefully. If no active record is found, the bot replies with a clear message asking the student to contact the department office or check again after the data is updated.

7.3 Exam Schedule Module

Exam Schedule Module is implemented as a focused service area within the chatbot. The module receives a detected intent, checks the logged-in student's context where required and returns information in a short response format suitable for chat display. Administrative maintenance screens keep the module data current.

Each module is designed to fail gracefully. If no active record is found, the bot replies with a clear message asking the student to contact the department office or check again after the data is updated.

7.4 Placement Updates

Placement Updates is implemented as a focused service area within the chatbot. The module receives a detected intent, checks the logged-in student's context where required and returns information in a short response format suitable for chat display. Administrative maintenance screens keep the module data current.

Each module is designed to fail gracefully. If no active record is found, the bot replies with a clear message asking the student to contact the department office or check again after the data is updated.

7.5 Event Notifications

Event Notifications is implemented as a focused service area within the chatbot. The module receives a detected intent, checks the logged-in student's context where required and returns information in a short response format suitable for chat display. Administrative maintenance screens keep the module data current.

Each module is designed to fail gracefully. If no active record is found, the bot replies with a clear message asking the student to contact the department office or check again after the data is updated.

7.6 Academic Support

Academic Support is implemented as a focused service area within the chatbot. The module receives a detected intent, checks the logged-in student's context where required and returns information in a short response format suitable for chat display. Administrative maintenance screens keep the module data current.

Each module is designed to fail gracefully. If no active record is found, the bot replies with a clear message asking the student to contact the department office or check again after the data is updated.

7.7 Administrator Module Details

The administrator module is essential because the chatbot is only as reliable as its stored data. The admin dashboard includes forms for adding exam records, placement drives, event notices and FAQ entries. Each form validates mandatory fields such as title, date, department and description before saving.

The dashboard also displays existing records in compact tables. Administrators can edit a record when dates change, mark notices as inactive after expiry and delete incorrect entries. For audit purposes, future versions can store the user who created each notice and the time of the last update.

7.8 Query Log and Analytics Module

The query log stores every message submitted by students along with detected intent and timestamp. This is not only useful for debugging but also for academic administration. If many students ask about placement eligibility, the training and placement cell may publish clearer instructions. If many students ask about attendance shortage, the department may schedule a briefing.

Analytics can be implemented as simple counts by intent, daily query volume and unresolved query percentage. These metrics help the system improve over time because administrators can add new FAQ entries for questions that currently receive fallback responses.

CHAPTER 8: DATABASE DESIGN

The database design stores academic data in normalised tables. Personalised data such as attendance is linked to the student roll number, while general notices such as exams, placements and events are stored as active records with dates and descriptions.

8.1 User Table Schema

Field	Type	Description
user_id	INTEGER PK	Unique user identifier
roll_no	VARCHAR(20)	Student roll number
name	VARCHAR(80)	Full name of the user
email	VARCHAR(100)	Registered email address
password_hash	TEXT	Hashed password
role	VARCHAR(20)	student or admin
department	VARCHAR(50)	Department name

8.2 Attendance Table Schema

Field	Type	Description
attendance_id	INTEGER PK	Unique attendance record
roll_no	VARCHAR(20)	Student roll number
subject_code	VARCHAR(20)	Subject identifier
subject_name	VARCHAR(80)	Name of subject
classes_held	INTEGER	Total conducted classes
classes_attended	INTEGER	Classes attended by student
percentage	DECIMAL	Calculated attendance percentage

8.3 Exam Schedule Table Schema

Field	Type	Description
exam_id	INTEGER PK	Unique exam record
exam_type	VARCHAR(30)	Internal, practical or semester
subject	VARCHAR(80)	Subject name
exam_date	DATE	Scheduled date
time_slot	VARCHAR(40)	Exam timing
room_no	VARCHAR(20)	Room or lab number

8.4 Placement and Event Table Schemas

Table	Key Fields	Purpose
placements	company_name, eligibility, package, drive_date, venue	Stores placement drive announcements.

events	event_name, category, event_date, coordinator, venue	Stores campus event notifications.
faqs	question_pattern, answer, intent	Stores reusable academic support answers.
query_logs	user_id, query_text, intent, response_time	Stores chatbot usage history.

8.5 Entity Relationship Summary

Relationship	Description
users to attendance	One student may have many attendance records.
users to query_logs	One user may generate many chatbot queries.
admin to notices	Administrators create or update exam, placement and event records.
intent to response	Detected intents map to database queries or FAQ answers.

8.6 Normalisation and Data Integrity

The schema follows a normalised structure so that repeated data is minimised. User details are stored in the users table, while attendance, query logs and support interactions refer to the student by identifier or roll number. General notices are separated by domain because exam schedules, placement updates and events have different fields.

Data integrity is maintained through primary keys, controlled input types and application-level validation. For example, exam dates must be valid dates, attendance percentages should be calculated from classes held and attended, and placement records should include eligibility criteria before they are made active.

8.7 Sample Data Records

A sample attendance record may contain roll number 24N81A6261, subject code CS201, subject name Data Structures, classes held 42, classes attended 38 and percentage 90.48. When the student asks for attendance, the chatbot groups all subject records and returns the percentage in readable form.

A sample placement record may contain company name TechNova Solutions, role Graduate Trainee, eligibility 70 percent and above, drive date 12 March 2026 and venue Seminar Hall. When the student asks for placement updates, the bot lists active drives sorted by upcoming date.

A sample event record may contain event name CodeSprint, category Technical, coordinator name, venue, registration deadline and event date. This structure allows the same chatbot response format to work for workshops, seminars, fests and sports events.

CHAPTER 9: IMPLEMENTATION

Implementation begins with the Flask application structure. Separate folders are maintained for templates, static files, routes, models and configuration. The backend exposes normal page routes for login and administration and an API route for chatbot messages.

9.1 Flask Application Flow

- The application object is created with configuration for secret key and database connection.
- Database models define users, attendance, exams, placements, events, FAQs and query logs.
- The login route validates credentials and creates a secure session.
- The `/chat` route accepts JSON input, calls the intent detector and returns a JSON response.
- Admin routes implement create, read, update and delete operations for institutional records.

9.2 Intent Detection

The initial implementation uses keyword matching and pattern groups. Queries containing words such as attendance, percentage or shortage are mapped to the attendance intent. Queries containing exam, timetable, schedule or hall are mapped to the exam intent. Placement, company, drive and eligibility map to placement updates, while event, fest, workshop and registration map to event notifications.

9.3 Frontend Implementation

The frontend uses a chat panel with message bubbles, input field and send button. JavaScript prevents page reload, sends the typed query through fetch, displays a loading state and appends the bot response. CSS media queries ensure that the same interface works on mobile screens.

9.4 Administrator Implementation

The administrator dashboard provides compact tables for editing data. Each record contains edit and delete actions. Forms are validated on both client and server sides to avoid incomplete schedules or notices.

9.5 Error Handling

The system returns controlled messages for invalid login, missing records, database errors and unrecognised queries. This avoids exposing internal errors to students and keeps the chatbot experience professional.

9.6 Implementation of Attendance Responses

Attendance responses are generated by filtering records with the logged-in student's roll number. The backend calculates or reads the percentage and formats each subject as a short line. If any subject falls below the required threshold, the response highlights that the student should contact the class in-charge or improve attendance before eligibility verification.

This module is personalised and therefore requires authentication. A student cannot request another student's attendance simply by typing a roll number if the backend is configured to use the session roll number. This protects student privacy while still making the feature convenient.

9.7 Implementation of Notice Responses

Exam, placement and event responses are treated as notice-style data. The backend fetches only active records and sorts them by date. The response includes the title, date, time or venue and a brief description. Long notices are summarised so that the chat window remains readable.

Expired records can be retained in the database for history while hidden from student responses. This approach is better than deleting old notices because administrators may need to review past schedules or placement announcements.

9.8 Implementation Challenges

One challenge is handling different forms of the same question. A student may type 'attendance', 'my percentage', 'shortage details' or 'am I eligible for exam'. Keyword groups reduce this issue, but future natural language processing can improve coverage. Another challenge is keeping the data updated; the admin module is therefore designed to make updates quick.

Responsive design is also important because students may use small screens. Tables and long notices must be converted into compact chat-friendly messages. The frontend implementation avoids large fixed-width elements in the student view.

CHAPTER 10: TESTING

Testing verifies whether the College Chat Bot meets functional and non-functional requirements. Manual and module-level test cases were prepared for authentication, chatbot responses, administration, database operations and responsive interface behaviour.

10.1 Authentication Module Test Cases

Test ID	Scenario	Expected Result	Status
TC-A01	Login with valid student credentials	Student dashboard opens	Pass
TC-A02	Login with invalid password	Error message displayed	Pass
TC-A03	Access admin URL as student	Access denied	Pass
TC-A04	Logout from active session	Session cleared	Pass

10.2 Chatbot Query Test Cases

Test ID	Input Query	Expected Response	Status
TC-C01	What is my attendance?	Subject-wise attendance percentage	Pass
TC-C02	When is the next exam?	Upcoming exam schedule	Pass
TC-C03	Any placement updates?	Latest active placement drives	Pass
TC-C04	Show college events	Upcoming event notifications	Pass
TC-C05	Unknown random question	Fallback support response	Pass

10.3 Administration Module Test Cases

Test ID	Scenario	Expected Result	Status
TC-AD01	Create exam schedule	Record saved and visible to chatbot	Pass
TC-AD02	Update placement drive	Updated details returned	Pass
TC-AD03	Delete event notice	Event removed from active responses	Pass
TC-AD04	Submit incomplete form	Validation message shown	Pass

10.4 Responsive Interface Testing

The interface was tested on desktop and mobile viewport sizes. The chat window, input field, navigation buttons and administrator tables remain readable without horizontal scrolling on standard mobile screens.

10.5 Database Testing

Database testing ensures that records are inserted, retrieved, updated and deleted correctly. The attendance module was checked with multiple subjects for the same roll number. The exam, placement and event modules were checked with active and inactive records to verify that students see only current information.

Input validation was tested using missing dates, blank titles and invalid percentage values. The system should reject incomplete records and show useful messages to the administrator rather than saving unreliable data.

10.6 Usability Testing

Usability testing focused on whether a student can understand the chatbot without separate instructions. The interface was checked for clear message bubbles, visible send button, readable response text and proper handling of empty messages. The fallback response was also tested to ensure that unsupported questions do not appear as system failures.

Administrator usability was tested by adding sample exams, placement drives and events. The forms were reviewed for compactness, field order and clarity. A good administrator experience is necessary because outdated data would reduce student trust in the chatbot.

10.7 Security Testing

Security testing verified that administrator routes are not available to normal students. Login sessions were checked by attempting to access protected pages after logout. Basic input handling was also tested by submitting special characters in chat queries and form fields.

The current prototype can be strengthened further with password complexity checks, CSRF protection for forms, HTTPS deployment and role-specific database privileges. These improvements are recommended before production deployment.

CHAPTER 11: RESULTS

The College Chat Bot successfully provides instant answers for the selected academic domains. The system reduces repeated manual enquiries and improves student access to updated information. Query logs also provide insight into the most common student needs.

Area	Tested Cases	Passed	Pass Percentage
Authentication	4	4	100%
Chatbot Queries	5	5	100%
Administration	4	4	100%
Database Operations	6	6	100%
Responsive UI	4	4	100%

Parameter	Manual Enquiry Process	College Chat Bot
Response time	Depends on staff availability	Instant for available records
Availability	Limited to office hours	Accessible through web interface
Consistency	May vary by source	Uses central database
Record keeping	Usually not maintained	Query logs stored
Student convenience	Requires office visit or messages	Single chat window

11.1 Sample Response Behaviour

For an attendance query, the chatbot returns a subject-wise summary and warns the student if the attendance percentage is below the required eligibility threshold. For a placement query, the bot lists active company drives with date, venue and eligibility information.

11.2 Observed Improvements

The most visible improvement is response speed. Common information that previously required a student to message a coordinator or visit an office can be retrieved immediately. This does not remove the need for official notices, but it gives students a dependable self-service channel.

The second improvement is consistency. Since answers are generated from database records, all students receive the same current exam schedule or placement notice. This reduces misunderstandings caused by copied messages or incomplete screenshots.

The third improvement is maintainability. Staff members can update data through forms, and the chatbot begins using the new record without code changes. This makes the project practical for repeated academic use.

11.3 Limitations of the Current Version

The current version is primarily rule-based. It can answer well-defined query categories but may not understand highly complex sentences. The bot also depends on manually updated data unless integrated with an institutional ERP. These limitations are acceptable for a prototype but should be addressed in future versions.

Another limitation is that notifications are pull-based: students ask the bot and receive information. Push notifications through email, SMS or mobile alerts would make the system more proactive for urgent announcements.

CHAPTER 12: ADVANTAGES

- Provides a single point of access for common college information.
- Reduces repeated manual enquiries for faculty and office staff.
- Improves communication of exam schedules, placement updates and events.
- Supports responsive access from mobile and desktop browsers.
- Stores structured data that can be updated by authorised administrators.
- Maintains query history for identifying recurring student concerns.
- Uses open-source technologies that are easy to deploy and maintain.

The system is also scalable at the design level. Additional modules such as library dues, bus routes, fee reminders and timetable lookup can be added without changing the overall architecture.

12.1 Institutional Advantages

For the institution, the chatbot creates a controlled information channel. Administrators can ensure that important details such as exam dates and placement eligibility are published in a structured format. Departments can also use query logs to improve student communication.

The project is cost-effective because it uses open-source technologies. A small college department can deploy the prototype on existing infrastructure or low-cost hosting without purchasing a large ERP product.

12.2 Student Advantages

Students benefit from immediate access and reduced uncertainty. They can verify attendance, schedules and notices outside office hours. The responsive interface allows the system to be used during travel, at home or between classes.

The chatbot format is less intimidating than formal portals for simple questions. Students can ask direct questions and receive a short answer instead of searching through many menu items.

CHAPTER 13: FUTURE SCOPE

Future versions of College Chat Bot can include machine-learning-based natural language understanding so that the bot recognises a wider variety of student expressions. A trained intent classifier can improve accuracy beyond keyword matching.

Multilingual support can help students ask questions in regional languages. Voice input and text-to-speech responses can improve accessibility for users who prefer spoken interaction. Integration with college ERP or LMS platforms can make attendance and timetable data fully automatic.

Push notifications can be added for exam reminders, placement registration deadlines and emergency announcements. Analytics dashboards can show administrators the most frequent query categories, unresolved questions and peak usage periods.

- AI-based intent classification and entity extraction.
- ERP/LMS integration for live attendance and timetable synchronisation.
- Mobile application or progressive web app deployment.
- Email, SMS and WhatsApp notification integration.
- Multilingual and voice-enabled chatbot interaction.
- Advanced analytics dashboard for student query trends.

13.1 Integration Roadmap

The first future integration should connect the attendance module with the official student database or ERP. This would eliminate manual entry and make attendance responses live. The second integration should connect the examination cell schedule so that revised timetables are reflected immediately.

The training and placement cell can be given a separate role with permission to update only placement records. Similarly, event coordinators can update event notices without accessing attendance data. This role separation would make the system safer for institutional use.

13.2 Intelligent Chatbot Enhancements

Machine-learning models can improve intent detection by learning from query logs. Entity extraction can identify dates, subjects, company names and event categories from student messages. A hybrid design can combine machine learning with database rules so that responses remain accurate and auditable.

The system can also support multilingual responses. In a diverse student environment, English can remain the default while selected common responses are translated into regional languages.

CHAPTER 14: CONCLUSION

College Chat Bot demonstrates how a compact Flask-based web application can improve academic communication within a college. By combining a responsive chat interface, structured database records and administrator-maintained content, the system provides quick answers to common student queries.

The project covers attendance information, exam schedules, placement updates, event notifications and academic support. Testing shows that the system can handle the required workflows with consistent results. The solution reduces manual effort, increases information availability and creates a foundation for future intelligent student support services.

Overall, the project satisfies the objective of building a practical, scalable and user-friendly chatbot system for college environments. With future integration and machine-learning enhancements, it can become a broader digital assistant for institutional academic services.

REFERENCES

1. Miguel Grinberg, Flask Web Development: Developing Web Applications with Python, O'Reilly Media.
2. Pallets Projects, Flask Documentation, <https://flask.palletsprojects.com/>
3. SQLite Documentation, <https://www.sqlite.org/docs.html>
4. MySQL Documentation, <https://dev.mysql.com/doc/>
5. Mozilla Developer Network, HTML, CSS and JavaScript Web Technology References.
6. R. S. Pressman, Software Engineering: A Practitioner's Approach, McGraw-Hill.
7. NIST, Role Based Access Control Model and Standard.
8. Bootstrap Documentation, <https://getbootstrap.com/docs/>
9. Python Software Foundation, Python 3 Documentation, <https://docs.python.org/3/>
10. Academic chatbot design references and educational technology survey articles.